

Hướng dẫn sử dụng Banking-Grade Copilot Package

Version: 2026-05-13 | **Scope:** `.github/` standalone package | **Target:** GitHub Copilot (VS Code, GitHub.com, CLI)

Mục lục

1. Tổng quan
2. Cài đặt vào project thực tế
3. Cách Copilot hoạt động tự động
4. Sử dụng Prompt Files
5. Sử dụng Custom Agents
6. Quy trình làm việc chuẩn
7. Chạy Governance Checks
8. Tài liệu dự án (`.github/docs/`)
9. Quy tắc bảo mật tuyệt đối
10. Cấu trúc thư mục
11. Format code sau khi AI gen

1. Tổng quan

Package này là bộ cấu hình GitHub Copilot **cấp ngân hàng**, hoàn toàn độc lập trong thư mục `.github/`. Mục đích: biến Copilot thành một senior engineer có kỷ luật:

Phân tích trước khi code

Luôn phân tích kiến trúc, luồng nghiệp vụ, blast radius trước khi đề xuất bất kỳ thay đổi nào.

Plan + Approval bắt buộc

Mọi thay đổi đều phải có plan được developer duyệt trước khi implement. Không tự code ngầm.

Review từng dòng

Sau implement phải review từng dòng thay đổi về correctness, security, privacy, data integrity.

Cập nhật tài liệu

Sau mỗi thay đổi phải cập nhật `.github/docs/` với evidence, risks, rollback notes.

Thành phần chính:

Loại	Số lượng	Mô tả
Instructions files	3	Quy tắc tự động theo loại file
Prompt files	22	Workflow gọi bằng <code>/tên</code>
Custom agents	10	Personas gọi bằng <code>@tên</code>
Agent skills	18	Domain knowledge tự động
Workflow	1	Governance checks thủ công
Docs base	7	Tài liệu dự án cập nhật liên tục

2. Cài đặt vào project thực tế

Bước 1 — Copy package:

```
# Từ repo này, copy toàn bộ .github/ sang target project
xcopy /E /I .github\ C:\path\to\your-project\.github\

# Hoặc trên Linux/Mac
cp -r .github/ /path/to/your-project/.github/
```

Bước 2 — Hardening bắt buộc cho target repo:

```
# Thêm CODEOWNERS (thay bằng team của bạn)
echo "* @your-team/banking-devs" > .github/CODEOWNERS

# Enable branch protection trên GitHub:
# Settings → Branches → Add rule → main
# ✓ Require pull request reviews before merging
# ✓ Require status checks to pass
# ✓ Restrict pushes that create files
```

Bước 3 — Bật GitHub Actions auto-trigger:

Mở [.github/workflows/manual-governance-checks.yml](#) và thêm trigger trên PR:

```
on:
  workflow_dispatch: # giữ nguyên
  pull_request:      # thêm dòng này
    branches: [main]
```

Lưu ý: Workflow hiện tại chỉ chạy thủ công (`workflow_dispatch`) — phù hợp cho package-only repo. Khi deploy sang project thật có code, cần bật `pull_request` trigger.

3. Cách Copilot hoạt động tự động

Không cần làm gì thêm — các rules tự động kích hoạt theo file đang mở:

Khi bạn làm việc với	Instructions tự động nhận
Bất kỳ file nào	Banking gates (copilot-instructions.md) + banking-grade rules
<code>.cs</code> , <code>.csproj</code> , <code>.razor</code> , <code>.cshtml</code>	ASP.NET Core patterns, DI, auth, Dapper/EF Core guidance
<code>.github/copilot/**</code> , <code>.github/docs/**</code> , <code>.github/skills/**</code>	Package maintenance rules

Skills tự động nhận theo context (không cần gọi tay):

► Xem danh sách 18 skills

4. Sử dụng Prompt Files

Gọi prompt trong VS Code Copilot Chat: gõ `/` rồi chọn tên prompt.

Nhóm hỏi & khám phá

Prompt	Khi nào dùng	Ví dụ
<code>/ask</code>	Hỏi câu hỏi kỹ thuật có context repo	<code>/ask tại sao AccountService dùng Unit of Work?</code>

Prompt	Khi nào dùng	Ví dụ
<code>/scout</code>	Tìm file, symbol, pattern trong codebase	<code>/scout</code> tất cả nơi gọi <code>TransferFunds</code>
<code>/explain-code</code>	Giải thích code hiện có và business flow	<code>/explain-code</code> + chọn đoạn code
<code>/analyze-code</code>	Phân tích kiến trúc, blast radius	<code>/analyze-code</code> module xác thực OTP

Nhóm lên kế hoạch

Prompt	Output	Khi nào dùng
<code>/plan</code>	Scope, affected files, phases, risks	Feature/fix thông thường
<code>/banking-plan</code>	+ Risk matrix, rollback, approval gates, verification commands	Mọi thay đổi ảnh hưởng DB, auth, PII

Quy tắc bắt buộc: Mọi thay đổi code phải có plan được duyệt trước. Copilot sẽ dừng ở bước plan và chờ xác nhận — không tự implement.

Nhóm thực thi

Prompt	Khi nào dùng
<code>/implement</code>	Implement sau khi plan đã được duyệt
<code>/fix</code>	Debug + trace root cause + fix minimal
<code>/logic-check</code>	Kiểm tra business logic, edge cases, integration behavior
<code>/test</code>	Tìm lệnh test phù hợp, chạy, phân tích failures

Nhóm review & tài liệu

Prompt	Khi nào dùng
<code>/review</code>	Review code changes với severity-ranked findings
<code>/line-review</code>	Review từng dòng — bắt buộc với banking-grade changes
<code>/docs-update</code>	Cập nhật project documentation
<code>/docs-base-update</code>	Cập nhật <code>./github/docs/</code> sau feature/fix
<code>/git</code>	Chuẩn bị commit message, PR description

Prompt	Khi nào dùng
<code>/mcp</code>	Chọn MCP tool phù hợp hoặc fallback thủ công
<code>/azure-devops-intake</code>	Đọc work item <code>dev.azure.com</code> đã approved và tạo plan inputs
<code>/db-schema-context</code>	Lấy schema metadata read-only, không đọc production rows
<code>/internal-knowledge</code>	Tra cứu coding convention, docs nội bộ, old project đã sanitize
<code>/deep-research</code>	Nghiên cứu nhiều nguồn có citation
<code>/architecture-research</code>	So sánh kiến trúc và tạo ADR-ready recommendation

Vòng lặp đầy đủ

```
/devloop [mô tả feature hoặc bug fix]
```

Chạy toàn bộ pipeline: **phân tích** → **plan** → **chờ approval** → **implement** → **test** → **review** → **docs**.

Không bỏ qua bước approval gate. `/devloop` sẽ dừng sau khi tạo plan và chờ developer xác nhận trước khi implement.

5. Sử dụng Custom Agents

Mở VS Code Copilot Chat → Agent mode → chọn agent từ dropdown.

Agent	Chuyên về	Gọi khi
System Analyst	Kiến trúc, luồng, dependencies, blast radius	Trước khi làm bất cứ thứ gì phức tạp
Planner	Plans chi tiết: phases, risks, approvals, rollback	Cần plan có đủ thành phần để duyệt
Researcher	Tìm và tổng hợp context trong repo/docs	Cần hiểu codebase trước khi plan
Debugger	Reproduce → trace root cause → minimal fix	Bug, lỗi CI, behavior bất thường
Tester	Tìm lệnh test, chạy, phân tích failures	Verify sau implement

Agent	Chuyên về	Gọi khi
Code Reviewer	Severity-ranked findings: Critical / High / Medium / Low	Review PR, sau implement
Security Reviewer	Banking security, privacy, data integrity, audit	Thay đổi auth, PII, DB
Docs Manager	Cập nhật docs, changelog, delivery log	Sau mọi thay đổi có nghĩa
Knowledge Curator	Curate docs nội bộ, source register, sanitized references	Khi thêm knowledge / NotebookLM-style source
Research Architect	Research kiến trúc, thư viện, ADR-ready output	Khi cần so sánh kiến trúc hoặc R&D

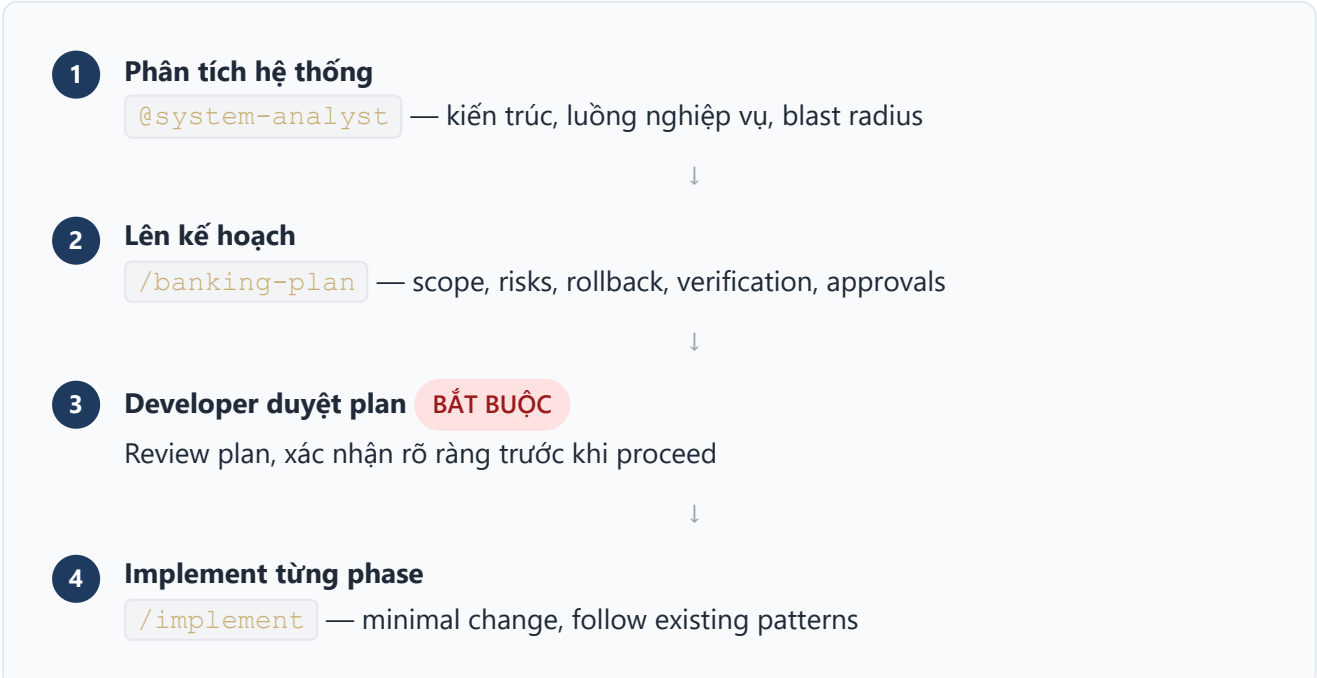
Ví dụ kết hợp agents:

```
# Bước 1: Hiểu hệ thống
@system-analyst phân tích luồng xử lý lệnh chuyển tiền và blast radius nếu đổi
schema bảng transactions

# Bước 2: Lên kế hoạch
@planner tạo banking-plan cho việc thêm cột audit_log vào bảng transactions

# Bước 3: Sau implement — review bảo mật
@security-reviewer review migration script vừa viết
```

6. Quy trình làm việc chuẩn



- ↓
- 4b

Format ngay sau khi AI gen code

Tự động

Copilot chạy ngay sau khi viết xong file `*.cs / *.razor / *.cshtml` :
`dotnet format <project> --include <chỉ-file-vừa-viết> --no-restore`
Chỉ format file đã chạm vào — không format toàn project
- ↓
- 5

Verify

`/test` — build, lint, test; `/fix` nếu failures
- ↓
- 6

Review code

`/line-review` — từng dòng; `@security-reviewer` cho sensitive changes
- ↓
- 7

Cập nhật tài liệu

`/docs-base-update` — project-docs-base, changelog, delivery-log
- ↓
- 8

Chuẩn bị PR

`/git` — commit message, PR description với evidence

7. Chạy Governance Checks

Chạy thủ công trên GitHub:

```
GitHub.com → repository → Actions tab
→ "Manual Banking Governance Checks"
→ Run workflow → chọn branch → Run
```

Các job sẽ chạy:

Job	Nội dung kiểm tra	Kết quả
Detect	Phát hiện project shape (.NET có hay không)	Metadata cho jobs sau
Validate Copilot package	Đủ required files, YAML frontmatter hợp lệ, removed assets không còn	Pass/Fail
Build + Test + Lint	<code>dotnet build</code> , <code>dotnet test</code> , <code>dotnet format --verify-no-changes</code>	Pass/Fail (skip nếu không có .NET)

Job	Nội dung kiểm tra	Kết quả
Secret Scan	Tìm private keys, GitHub tokens, AWS keys, password patterns	Pass/Fail
Dependency Audit	<code>dotnet list package --vulnerable --include-transitive</code>	CVE report
CodeQL SAST	Static analysis C# — security vulnerabilities	Alerts

Xem kết quả failed job:

```
gh run list --limit 10
gh run view <run-id> --log-failed
```

8. Tài liệu dự án (.github/docs/)

Ba file phải cập nhật sau mỗi thay đổi quan trọng:

File	Mục đích	Cập nhật khi
project-docs-base.md	Tài liệu sống: scope, kiến trúc, rules hiện tại	Thay đổi behavior, setup, architecture, API
project-changelog.md	Lịch sử thay đổi có ngày tháng	Mỗi feature/fix/migration
feature-delivery-log.md	Delivery log chi tiết với evidence, risks, rollback	Mỗi delivery có ý nghĩa

Cập nhật tự động bằng Copilot:

```
/docs-base-update [mô tả thay đổi vừa làm]
```

Copilot sẽ đọc context và cập nhật cả 3 file đúng format.

Template entry trong delivery log:

```
## [DATE] Feature/Fix Name

- **Goal**: ...
- **Scope**: files changed
- **Verification**: commands run + result
```


- **Risks**: residual risks
- **Rollback**: how to revert
- **Evidence**: test output / lint clean / build success

9. Quy tắc bảo mật tuyệt đối

Những điều KHÔNG BAO GIỜ được làm:

Hành động cấm	Lý do
Đưa data thật của khách hàng vào code/test/log/docs	Vi phạm banking data safety
Hardcode credentials, tokens, API keys bất kỳ đâu	Secret exposure risk
Dùng production data làm test data	PII/regulatory violation
Bỏ qua human review trước merge vào main	Banking control requirement
Log secrets, OTP, session IDs, private keys	Audit/security violation
Push force hoặc reset hard trên shared branches	Destructive action
Deploy/migrate DB production không có rollback plan	Irreversible risk

Copilot cũng sẽ từ chối:

- Tạo code giả lập / fake integration (simulate features)
- Làm yếu test để pass
- Bỏ qua validation tại trust boundaries
- Implement thay đổi DB/auth/encryption trước khi có explicit approval

10. Cấu trúc thư mục tham khảo nhanh

```
.github/
|
├─ copilot-instructions.md           ← Banking gates, LUÔN active với mọi
file
|
├─ instructions/
|   ├─ banking-grade-engineering.instructions.md  ← applyTo: ** (tất cả)
|   └─ aspnet-core.instructions.md               ← applyTo: *.cs, *.csproj,
...
└─ copilot-package-knowledge-base.instructions.md ← applyTo: .github/**
```

```

├── prompts/                                ← 22 prompts, gọi bằng /tên-prompt
│   ├── ask.prompt.md
│   ├── analyze-code.prompt.md
│   ├── explain-code.prompt.md
│   ├── plan.prompt.md
│   ├── banking-plan.prompt.md            ← Dùng cho mọi thay đổi banking-grade
│   ├── implement.prompt.md
│   ├── fix.prompt.md
│   ├── logic-check.prompt.md
│   ├── test.prompt.md
│   ├── review.prompt.md
│   ├── line-review.prompt.md            ← Bắt buộc trước mỗi PR
│   ├── docs-update.prompt.md
│   ├── docs-base-update.prompt.md
│   ├── scout.prompt.md
│   ├── git.prompt.md
│   ├── mcp.prompt.md
│   ├── azure-devops-intake.prompt.md
│   ├── db-schema-context.prompt.md
│   ├── internal-knowledge.prompt.md
│   ├── deep-research.prompt.md
│   ├── architecture-research.prompt.md
│   └── devloop.prompt.md                ← Full pipeline end-to-end
├── agents/                                ← 10 agents, chọn trong Agent mode
│   ├── system-analyst.agent.md
│   ├── planner.agent.md
│   ├── researcher.agent.md
│   ├── tester.agent.md
│   ├── debugger.agent.md
│   ├── code-reviewer.agent.md
│   ├── security-reviewer.agent.md
│   ├── docs-manager.agent.md
│   ├── knowledge-curator.agent.md
│   └── research-architect.agent.md
├── skills/                                ← 18 domain skills, tự động theo context
│   ├── banking-grade-engineering/
│   ├── system-analysis/
│   ├── codebase-reading/
│   ├── business-logic-analysis/
│   ├── code-solution-fit/
│   ├── planning-governance/
│   ├── secure-code-review/
│   ├── testing-verification/
│   ├── docs-base-maintenance/
│   ├── root-cause-debugging/
│   ├── backend-api-governance/
│   ├── database-data-integrity/
│   ├── release-devops-governance/
│   ├── aspnet-core-governance/
│   ├── dotnet-testing/
│   └── mcp-integration-governance/

```

```

├── internal-knowledge-governance/
├── deep-research-governance/
├── workflows/
│   ├── manual-governance-checks.yml    ← Chạy thủ công:
│   build/lint/test/secret/SAST
├── copilot/                            ← Knowledge base cho RAG retrieval
│   ├── README.md
│   ├── banking-grade-engineering.md
│   ├── workflow-playbook.md
│   ├── codebase-analysis-playbook.md
│   ├── azure-devops-mcp-playbook.md
│   ├── mcp-tool-registry.template.md
│   ├── internal-knowledge-playbook.md
│   ├── knowledge-source-register.template.md
│   ├── deep-research-playbook.md
│   ├── manual-tooling-guide.md
│   ├── copilot-architecture.md
│   ├── skills-index.md
│   ├── prompt-catalog.md
│   ├── agent-catalog.md
│   ├── official-docs-snapshot.md
│   ├── copilot-project-assessment.md
│   └── references/
│       ├── aspnet-core-service-patterns.md
│       ├── dotnet-data-integration-patterns.md
│       └── analysis-debug-review-patterns.md
├── docs/                               ← Tài liệu dự án (cập nhật liên tục)
│   ├── README.md
│   ├── project-docs-base.md
│   ├── project-changelog.md
│   ├── feature-delivery-log.md
│   ├── huong-dan-su-dung.md            ← File này
│   ├── ai-project-developer-guide.md
│   └── ai-project-presentation.html

```

11. Format code sau khi AI gen

Copilot **tự động format ngay** sau khi viết hoặc sửa bất kỳ file `.cs`, `.razor`, `.cshtml` nào — chỉ format những file vừa bị thay đổi, không đưng vào phần còn lại của project.

Quy tắc này được cài vào tất cả 3 lớp: `copilot-instructions.md`, `aspnet-core.instructions.md`, và từng prompt (`/implement`, `/fix`, `/devloop`). Copilot sẽ chạy format đúng thời điểm — ngay khi vừa viết xong file.

Lệnh Copilot chạy sau khi gen code

```
# Copilot sẽ tự chạy lệnh này sau khi viết bất kỳ .cs / .razor / .cshtml
dotnet format MyProject.sln --include src/Services/AccountService.cs --no-restore

# Nếu nhiều file:
dotnet format MyProject.sln --include src/Services/AccountService.cs
src/Controllers/AccountController.cs --no-restore
```

Quy tắc scope

Điều kiện	Hành động
AI vừa viết/sửa *.cs, *.razor, *.cshtml	Format ngay chỉ những file đó
File khác (.json, .md, .yaml, ...)	Không format
Toàn bộ project	Không bao giờ — chỉ format file đã chạm vào

Mối quan hệ với CI

Lớp	Công cụ	Hành động
Copilot (AI gen)	dotnet format --include <files>	Fix ngay — format khi AI vừa viết xong
CI workflow	dotnet format --verify-no-changes	Verify — fail build nếu code chưa format

Nếu có file nào chưa format lọt qua: CI sẽ bắt và block merge.

Package status: Production-ready | **Files:** 82 | **Broken references:** 0 | **Security issues:** 0 |

Last reviewed: 2026-05-13